VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



REPORT
COURSE: DISTRIBUTED SYSTEMS

STREAMING LAKEHOUSE DATA PLATFORM

SUPERVISOR(s): PhD. NGUYEN QUANG HUNG

STUDENT 1: NGO QUANG HIEU - 2311015
STUDENT 2: MAI THANH HOANG - 2311048
STUDENT 3: LE VU MINH HIEU - 2310953

——o0o——

HO CHI MINH CITY, Sep 2026

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of PhD. NGUYEN QUANG HUNG at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during my research.

Ho Chi Minh City, December 2027

Authors,

Ngo Quang Hieu

Acknowledgement

We would like to express my appreciation to Dr. NGUYEN QUANG HUNG for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

Besides, we are grateful for the help from Nguyen Thanh for valuable comments for research aspect and our proposed pipeline, including pointing out strengths as well as weaknesses of previous works on multimodal models so that we can elaborate on our ideas more effectively.

In addition, we would like to extend our gratitude to my family. Their unwavering faith in our abilities and constant encouragement have been my pillars of strength. Their belief in my potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

Abstract

This thesis presents *Streamify*, a project built to learn, hands-on, how a modern streaming platform works and to apply distributed-systems concepts to a real system, rather than studying them in the abstract. A continuous stream of synthetic music-streaming events is carried end to end: ingested with ordering and fault-tolerance guarantees, processed continuously as it arrives rather than on a batch schedule, landed into a storage layer that supports both transactional updates and analytical queries, transformed into an analytics-ready model, and visualized and monitored throughout – all under a small, fixed compute budget rather than assumed elastic scale.

Building this meant confronting, and resolving, the concrete distributed-systems problems that appear once a platform has more than one producer and consumer of the same data: partitioning and replication for ordered, fault-tolerant ingestion; checkpointed, exactly-once processing so a crash never silently loses or duplicates an event; a storage layer that serves frequent small writes and large analytical reads without one starving the other; and observability that shows whether the system is keeping up with its input, not just whether it is running.

Every major technology choice was made deliberately against concrete alternatives and the constraints of a single-machine deployment, rather than assumed as a given (Chapter 2). This thesis documents that reasoning alongside the resulting architecture (Chapter 4) and what building and operating the system revealed about how these distributed-systems concepts behave in practice (Chapter 5).

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Goals . . . 1
- 1.3 Scope . . . 2
- 1.4 Report Structure . . . 2

2 Related Work and Background **3**
- 2.1 Why Real-Time Processing Matters . . . 3
- 2.2 Message Queues and Publish-Subscribe Messaging . . . 4
- 2.3 Batch and Streaming Processing Engines . . . 5
- 2.4 Storage layer & Open Table Formats . . . 6
- 2.5 Medallion Layering . . . 7

3 Problem Statement **8**
- 3.1 The Problem Before Streaming Lakehouses . . . 8

4 Proposed Solution **10**
- 4.1 Architecture Overview . . . 10
- 4.2 Storage Layer . . . 10
- 4.3 Ingestion Layer . . . 12
- 4.4 Stream & Batch Processing Layer . . . 13
- 4.5 Transformation Layer . . . 14
- 4.6 Serving Layer . . . 15
- 4.7 Orchestration Layer . . . 15
- 4.8 Monitoring Layer . . . 16

5 Conclusion **17**
- 5.1 Results Achieved . . . 17
- 5.2 Limitations . . . 17
- 5.3 Future Directions . . . 17

References **17**

List of Figures

1 Pub-sub message queue decoupling producers and consumers. 4
2 Batch versus streaming (event-at-a-time and micro-batch). 5
3 Object storage and the table layer on top of it. 6
4 Open table format: copy-on-write versus merge-on-read. 6
5 Medallion architecture: bronze, silver, gold. 7

6 Batch ETL: extract, transform, load on a schedule. 8
7 Lambda architecture: batch and speed layers feed one serving layer. 9
8 Data swamp: unmanaged lake data is hard to trust. 9
9 Coupled (left) versus decoupled (right) compute and storage. 9

10 Streamify architecture overview. 10
11 Kafka cluster: 2 KRaft nodes, 3 partitions per topic, replication factor 2. 12
12 Spark cluster: driver, master, workers and executors. 13
13 Micro-batch model: the stream is cut into small Spark batch jobs. 13
14 Star schema: `fact_streams` and five dimensions. 14
15 dbt DAG: bronze sources, star schema, `wide_streams`. 14
16 Superset dashboard on `wide_streams`, queried via Spark Thrift Server. 15
17 `full_pipeline` DAG, every 30 min. The red run failed before the seed fix. . . 15
18 `spark_compact` DAG, every 6 h, one `compact` task. 15
19 Grafana Kafka dashboard: consumer lag and rate per topic. 16
20 Grafana Spark dashboard: job metrics, JVM heap, container CPU/memory. . . 16

Chapter 1

Introduction

In chapter 1, the overview, objectives, and goals of the research project are illustrated. The outline of the report is also presented.

1.1 Motivation

Modern data platforms increasingly need to react to events as they happen rather than waiting for the next batch load. A music-streaming service, for example, wants to know within seconds which songs are trending, whether a user's session just dropped, or whether ingestion is falling behind – questions a traditional overnight ETL job cannot answer. Building and operating such a platform correctly requires more than a single tool: it requires a message queue that can buffer and replay events, a stream-processing engine that can keep up with continuous arrival, a storage layer that supports transactional updates on top of cheap object storage, a transformation layer that turns raw events into trustworthy analytics tables, and a way to observe the whole system's health while it runs.

This project, *Streamify*, was motivated by the goal of building and demonstrating exactly such an end-to-end pipeline, using synthetic but realistic music-streaming events generated by Eventsim, rather than studying any single one of these components in isolation. Each stage of the pipeline – ingestion, stream processing, lakehouse storage, transformation, and visualization – was chosen and configured to work together as a single reproducible system, so that the trade-offs made at each stage (Chapter 2) could be evaluated in the context of the whole pipeline rather than as isolated technology choices.

A further motivation was to explore these technologies under realistic resource constraints. Rather than assuming unlimited cloud compute, the project deliberately targets a small, single-machine deployment (Section 4.4), so that decisions such as Kafka cluster sizing and Spark cluster sizing (Section 4.4) had to be made under the same resource pressure a small team or a proof-of-concept deployment would face in practice.

1.2 Goals

The goals of this project are:

- **Build a working, end-to-end streaming lakehouse pipeline, and learn how a modern streaming platform works.** Take synthetic music-streaming events from generation (Eventsim) through ingestion (Kafka), stream processing (Spark Structured Streaming), lakehouse storage (Hudi), transformation (dbt), and visualization (Superset), as a single reproducible system rather than isolated proofs of concept.
- **Apply the distributed-systems concepts from the course to a real system.** Exercise partitioning, replication, checkpointed exactly-once processing, and consumer-lag monitoring, so the pipeline shows the behaviors that distinguish a streaming design from a periodic batch job (Section 2.4).
- **Make the whole system observable and reproducible.** Provide Prometheus/Grafana monitoring of Kafka lag and Spark resource usage, and deploy the entire stack with a single Docker Compose command.

1.3 Scope

This project covers the full path from synthetic event generation to dashboarding: Eventsim, Kafka ingestion, Spark Structured Streaming into Hudi bronze tables, dbt transformation into a star schema and wide table, Superset dashboards, and Prometheus/Grafana monitoring (Chapter 4). It also covers the distributed-systems concerns of the course – partitioning, replication, checkpointed exactly-once processing, and consumer-lag monitoring (Section 2.4). Everything is deployed with Docker Compose on a single machine.

Out of scope:

- **Production security hardening.** Authentication, network isolation, secrets management, and TLS are not addressed; the stack is for local demonstration.
- **Multi-region or high-availability deployment.** The pipeline targets one machine and one region (Section 4.4); failover and autoscaling are not implemented.
- **Real user data.** All events are synthetic, generated by Eventsim from the 10,000-song Million Song Dataset subset (Chapter 3).

1.4 Report Structure

- **Chapter 2** covers the background theory: pub-sub messaging, Spark's micro-batch model, Hudi table formats, and data modeling.
- **Chapter 3** describes the problems that batch ETL, lambda architectures and plain data lakes had before streaming lakehouses.
- **Chapter 4** presents the architecture, the implementation workflow, and the environment setup.
- **Chapter 5** summarizes the results, limitations, and future work.

Chapter 2

Related Work and Background

This section introduces the general concepts behind a streaming lakehouse – message queues, stream processing engines, open table formats, and dimensional modeling – independent of any one product. Chapter 4 then explains which concrete tool implements each concept in this project, and why that tool was chosen over its alternatives.

2.1 Why Real-Time Processing Matters

A modern platform such as a music-streaming service does not produce data in daily batches: it produces a continuous stream of small events – a song played, a page viewed, a login – the instant a user acts. Streamify, the project this report builds, simulates exactly this kind of platform with Eventsim, which generates synthetic "song played", "page view", and "authentication" events for a fake music website at a steady, ongoing rate.

In a traditional batch-processing setup, these events would simply accumulate in a source system and be pulled into the warehouse on a fixed schedule – once an hour, once a day. Between runs, every dashboard and every downstream decision is working from data that is, on average, half of that interval out of date; right before the next run, it is a full interval out of date. For a platform that wants to react to what users are doing now – surfacing trending songs, flagging a spike in failed logins, feeding a recommendation engine – that lag is the problem: the data is correct, just always late.

Real-time (streaming) processing closes this gap: instead of waiting for a scheduled run, each event is picked up and made queryable within seconds to minutes of happening, so dashboards and downstream models reflect what is happening now rather than what happened last interval. In Streamify, this means a play event lands in its Kafka topic within milliseconds of Eventsim emitting it, and is queryable as a Hudi bronze row one Spark Structured Streaming micro-batch later (Figure 2) – typically a few seconds – rather than at the next hourly or daily load.

Doing this reliably, and not just quickly, is harder than it sounds once the platform has more than one producer and more than one consumer of the same data. Four concrete problems show up:

- **Ordering.** Events from different partitions of the same topic can arrive and be processed out of order relative to each other.

- **Fault tolerance.** Any component in the chain – broker, processing job, storage layer – can crash mid-stream and must be able to resume without losing data.
- **Exactly-once delivery.** On a retry after a crash, the same event must not be silently duplicated or dropped.
- **Schema drift.** Upstream event fields change shape over time (added, renamed, or re-typed), and downstream tables must tolerate that without breaking.
- **Read/write contention.** The storage layer must serve a constant stream of small writes and large analytical reads at the same time, without one starving the other.

These are the distributed-systems problems that the rest of this section's concepts exist to solve: a *message queue* for ordered, fault-tolerant ingestion; a *stream processing engine* for continuous, exactly-once processing; and an *open table format* for a storage layer that supports both frequent streaming writes and fast analytical reads.

2.2 Message Queues and Publish-Subscribe Messaging

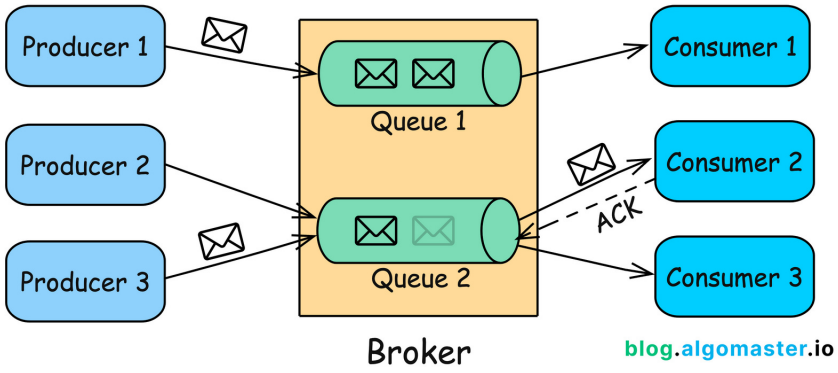


Figure 1: Pub-sub message queue decoupling producers and consumers.

Event streaming treats data as an unbounded sequence of immutable events (e.g. "user played a song") rather than as static rows in a table. A *message queue* built for this pattern is typically a log-based, publish-subscribe (pub-sub) broker, shown in Figure 1: producers append events to a durable, ordered log rather than pushing directly to a fixed receiver, and publish them under named *topics*; one or more consumers subscribe to the topics they need and read the log independently, at their own pace. This decouples producers from consumers in both time and number – a producer does not need a consumer to be running or fast enough to keep up, and any number of independent consumers can read the same retained events without the broker deleting them on delivery.

2.3 Batch and Streaming Processing Engines

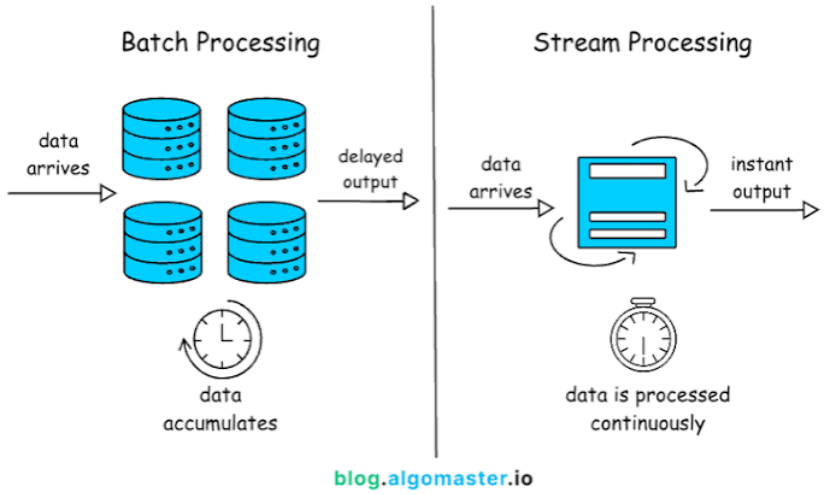


Figure 2: Batch versus streaming (event-at-a-time and micro-batch).

A *batch processing engine* operates on a bounded, complete dataset that is already at rest: it reads the full input, runs the job once end to end, produces its output, and stops until the next scheduled run.

A *stream processing engine* instead operates on an unbounded dataset that keeps arriving: it stays running indefinitely, continuously reading new events from a message queue, applying the same kind of transformation logic (filtering, aggregation, joins) to them, and writing results incrementally as they are produced, rather than waiting for a full batch to accumulate first. Figure 2 contrasts the two.

A stream processing engine can still process its unbounded input using either of two execution models:

- **Event-at-a-time.** Each event is processed as soon as it arrives, giving the lowest possible per-event latency.
- **Micro-batching.** The engine groups whatever new data has arrived since the last run into a small, discrete batch job on a fixed trigger interval, and processes that batch with the same execution engine used for ordinary batch queries. This reuses a mature batch engine's query planning and fault tolerance, at the cost of end-to-end latency being bounded by the trigger interval rather than being instantaneous.

Either execution model still reads its input positions (offsets) from the upstream message queue, and both keep the engine running continuously – the distinction from batch processing is not how the input is chopped up, but that the engine never stops waiting for the next event.

2.4 Storage layer & Open Table Formats

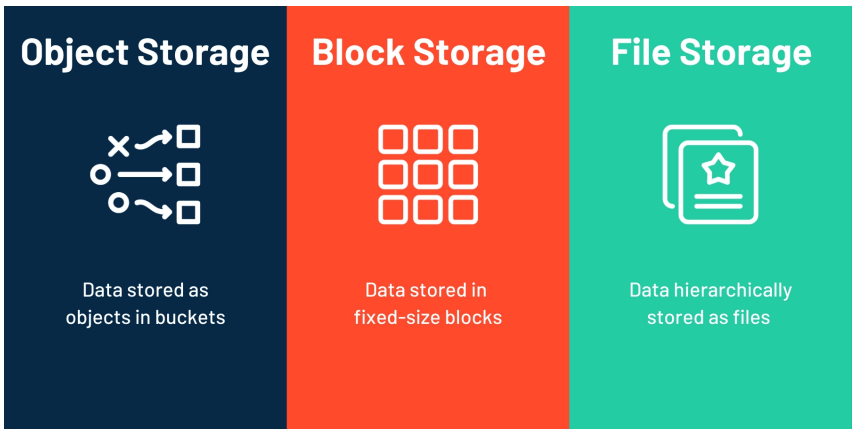


Figure 3: Object storage and the table layer on top of it.

As Figure 3 shows, a lakehouse's underlying storage is one of three kinds:

- **Object storage.** Data is stored as whole objects in flat buckets, addressed by key rather than by a filesystem path (e.g. S3, MinIO). This is the layer a lakehouse is built on.
- **Block storage.** Data is stored in fixed-size blocks that an operating system assembles into a filesystem, as used by a traditional database's local disk.
- **File storage.** Data is stored hierarchically as files in a shared, path-addressed filesystem (e.g. HDFS, NFS).

A lakehouse is built on object storage because it is cheap, horizontally scalable, and durable, but plain object storage is not a database: it does not provide row-level insert/update/delete semantics, ACID transactions, or time-travel queries – it only stores and returns whole objects. An *open table format* adds those capabilities on top of plain files in object storage, so a lake of files behaves like a table rather than an unmanaged pile of Parquet.

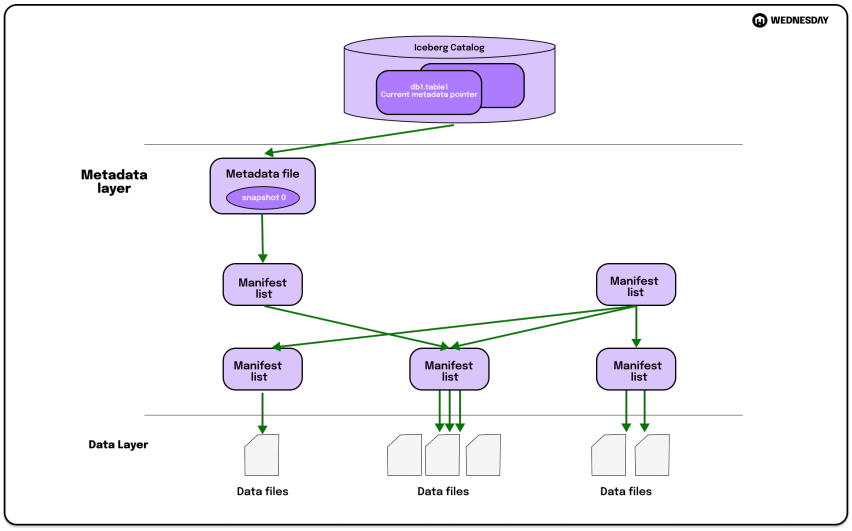


Figure 4: Open table format: copy-on-write versus merge-on-read.

Open table formats typically support two write strategies that trade off write cost against read cost, as summarized in Figure 4:

- **COPY_ON_WRITE (COW).** Every write rewrites the affected data files as new versioned files. Reads are fast (plain columnar files, no merge step), but writes are more expensive because entire files are rewritten on update.
- **MERGE_ON_READ (MOR).** Writes are appended to a compact row-based delta log next to the existing base files; the two are merged only at read time (or during a background compaction job). Writes are cheap and low-latency, at the cost of extra work on read (or a periodic compaction step) to materialize the merged view.

MOR suits high-frequency streaming appends, where write latency matters most; COW suits tables that are rebuilt in full on a schedule and then read many times, where read performance matters most.

2.5 Medallion Layering

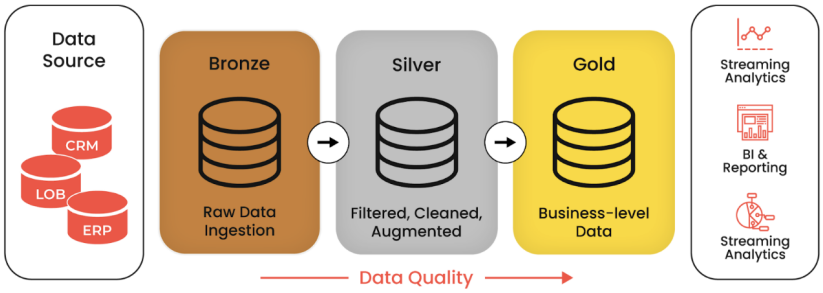


Figure 5: Medallion architecture: bronze, silver, gold.

The medallion architecture, shown in Figure 5, organizes a lakehouse into three successive layers of increasing refinement: bronze, silver, and gold.

- **Bronze** tables hold raw or lightly-cleaned data close to its source form, written as-is by the ingestion layer.
- **Silver** tables hold data that has been cleaned, deduplicated, and conformed to a consistent schema, but not yet aggregated or reshaped for a specific consumer.
- **Gold** tables hold data that has been modeled and aggregated into the shape consumers (BI tools, dashboards) actually query, typically built by a transformation layer reading from silver (or, when a project skips silver, directly from bronze).

This project uses all three layers: dbt reads bronze and builds the five dimension tables as silver, then reads bronze together with those silver dimensions to build the gold `fact_streams` and `wide_streams` tables (Section 4.5).

Chapter 3

Problem Statement

3.1 The Problem Before Streaming Lakehouses

Before message queues, stream engines, and open table formats were combined into one pattern (Chapter 2), teams that wanted both fresh data and a queryable, transactional lake had to work around four gaps.

3.1.1 Latency of Batch ETL

As Section 2.1 covers, a warehouse fed by scheduled batch loads is always behind reality: dashboards reflect the last load, not the current state. This was accepted because scheduled ETL had no continuous alternative. Figure 6 shows the pattern: many sources are extracted, transformed, and loaded in one scheduled pass.

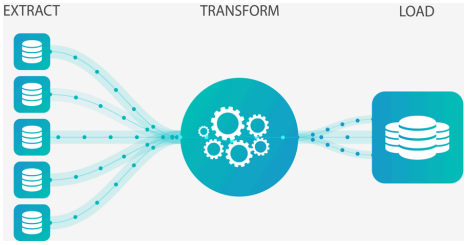


Figure 6: Batch ETL: extract, transform, load on a schedule.

3.1.2 Lambda-Architecture Complexity

The historical fix was the *lambda architecture*: a batch layer computing complete results on a schedule, plus a separate real-time *speed layer* for low-latency results in between. The catch is two systems and two codebases implementing the same business logic. Every transformation is written twice and the two can drift apart, a recurring source of subtle bugs. Figure 7 shows the two parallel paths that meet in a serving layer.

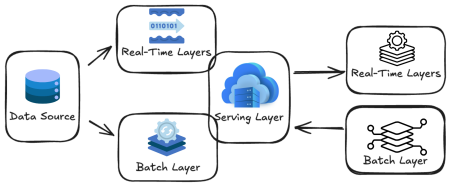


Figure 7: Lambda architecture: batch and speed layers feed one serving layer.

3.1.3 No Serving Layer on the Data Lake

Plain object storage with raw files (Parquet on S3 or HDFS, without an open table format) has no row-level updates, no ACID transactions, and no time travel (Section 2.4). The lake was an append-only dump, unusable as a serving layer. Workloads needing transactions had to go through a separate warehouse, adding a second copy of the data and another ETL hop. Without control over what lands in the lake, it turns into a *data swamp* (Figure 8).

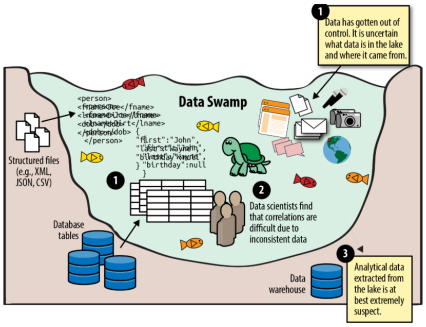


Figure 8: Data swamp: unmanaged lake data is hard to trust.

3.1.4 Storage and Compute Coupled in the Warehouse

Traditional warehouses bundle storage and compute, so scaling one means paying for the other. Together with the previous point, teams kept two copies of the data (raw in the lake, structured in the warehouse), doubling storage cost and sync work. Figure 9 shows the fix: compute and storage scale on their own.

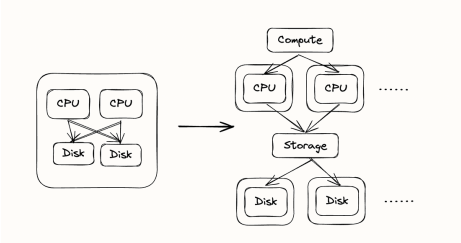


Figure 9: Coupled (left) versus decoupled (right) compute and storage.

A stream engine plus an open table format (Section 2.4) closes these gaps: bronze tables on object storage stay cheap, gain COW/MOR transactional writes, and are fed continuously – the *streaming lakehouse* this report builds.

Chapter 4

Proposed Solution

4.1 Architecture Overview

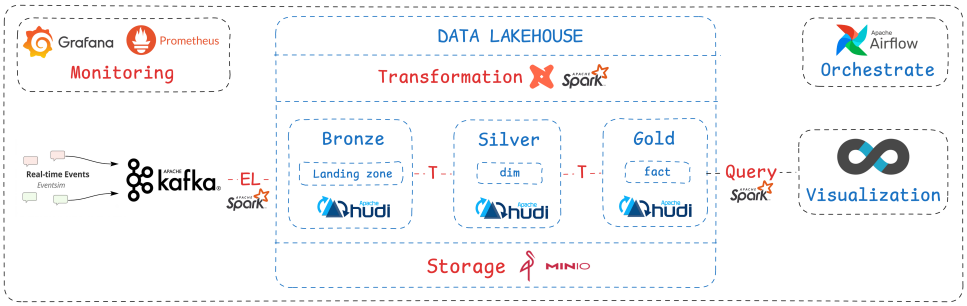


Figure 10: Streamify architecture overview.

Figure 10 shows the end-to-end architecture: Eventsim generates events into a Kafka [2] cluster, Spark Structured Streaming [4] consumes and lands them as Hudi [1] bronze tables, dbt transforms the bronze layer into silver dimension tables and gold fact/wide tables on the same Spark cluster, Superset visualizes the result through the Spark Thrift Server, and Airflow schedules the dbt, dashboard-refresh and compaction jobs, alongside a Prometheus/Grafana monitoring stack.

The rest of this chapter walks through the layers one by one, then the deployment. Each layer ends with a short note on why the tool was chosen (see Chapter 2).

4.2 Storage Layer

Tables are Hudi-managed, backed by MinIO [6] (S3-compatible) via the Hadoop S3A connector. There is no separate catalog service: each table is self-describing at its storage path (a `.hoodie/` metadata directory) and is (re-)registered into Spark's catalog idempotently at job startup. Bronze tables use the MERGE_ON_READ table type, so streaming writes are cheap and any delta log files are merged into columnar base files by a scheduled compaction job (Section 4.7); silver, gold, and seed tables use COPY_ON_WRITE, where dbt merges new rows into the existing files.

Layer	**Tool**	**Role**
Data simulation	Eventsim	Generates synthetic music-streaming events based on the Million Song Dataset
Message queue	Apache Kafka 4.3 (KRaft)	2-node cluster, replication factor 2, 3 partitions per topic, no ZooKeeper
Stream processing	Spark Structured Streaming 3.5.3	Reads Kafka, decodes/enriches events, writes bronze tables to Hudi
Table format / storage	Apache Hudi / Minio	MERGE_ON_READ bronze streaming tables, COPY_ON_WRITE silver/gold/seed tables, on MinIO/S3 via the Hadoop S3A connector
Compute cluster	Spark Standalone (1 master, 2 workers)	Shared by the streaming job and dbt (via Thrift), 4 cores total
Transformation	dbt (`dbt-spark`, `method: thrift`)	SQL-based star schema and wide-table modelling, incremental `merge` by key, submitted via the Spark Thrift Server
Orchestration	Apache Airflow 3	Runs dbt and the Superset refresh every 30 minutes and Hudi compaction every 6 hours, one job at a time through the Thrift Server
Visualization	Apache Superset	Dashboards connected to the Spark Thrift Server
Monitoring	Prometheus, Grafana	Kafka consumer-group lag, Spark JVM/streaming metrics, per-container CPU/memory
Containerization	Docker Compose	Single-command reproducible deployment

Table 4.1: Technology stack per pipeline layer.

Why Hudi, and why MOR bronze / COW silver and gold. Hudi's self-describing `.hoodie/` metadata avoids the extra catalog or warehouse service that plain Parquet or Snowflake would require. Bronze uses MOR to keep frequent streaming appends cheap; silver and gold use COW because they are written in batches by dbt and then read repeatedly – by downstream dbt models or by Superset. The bronze writer currently uses Hudi's `insert` operation, which adds new Parquet files and no log files, so compaction finds nothing to merge until updates or deletes reach a bronze table.

4.3 Ingestion Layer

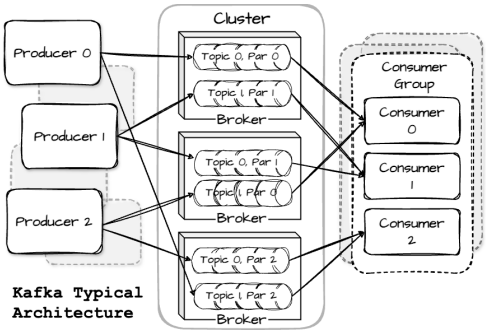


Figure 11: Kafka cluster: 2 KRaft nodes, 3 partitions per topic, replication factor 2.

Eventsim simulates a real-world user base, generating page-view and song-play events that mimic activity on a music-streaming website. Apache Kafka 4.3 runs in KRaft mode (no ZooKeeper) as a 2-node cluster (`broker-1`, `broker-2`). Each of the three event topics – `listen_events`, `page_view_events`, `auth_events` – has 3 partitions and replication factor 2, so every partition has a replica on both brokers, as shown in Figure 11.

Because KRaft mode is used, each broker itself also acts as a metadata/controller node, removing the need for a separate ZooKeeper ensemble. For each topic, the 3 partitions are spread across the two brokers, and because replication factor is 2, every partition is fully replicated: one broker holds the leader replica for a given partition and the other holds the follower replica, and the two brokers swap this role across partitions so that neither broker is leader for every partition of a topic. Producers (Eventsim) and consumers (the Spark Structured Streaming job) always read from and write to the leader replica of a partition; the follower replica only pulls data from the leader to stay in sync, and takes over as leader if its broker fails. This is what gives the cluster broker-level fault tolerance despite running on just two nodes.

4.4 Stream & Batch Processing Layer

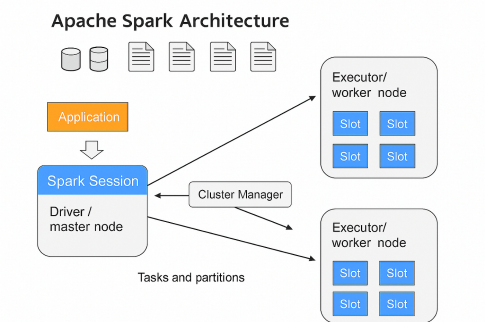


Figure 12: Spark cluster: driver, master, workers and executors.

The whole stack runs on one machine from a single `docker-compose.yml`. Its Spark Standalone cluster, shown in Figure 12, has 1 master and 2 workers with 2 cores / 2 GB each (4 cores in total), and runs both the streaming job and dbt's batch queries; the streaming job and the Thrift Server take 2 cores each. A *driver* process plans a job and schedules its tasks; the *master* tracks which workers are free and grants the driver executor slots on them; each *worker* then launches one or more *executor* processes that run the driver's tasks and cache data. The streaming job's driver is submitted once and keeps its executor slots for the life of the pipeline, while dbt submits short-lived batch queries through the Thrift Server on the same cluster.

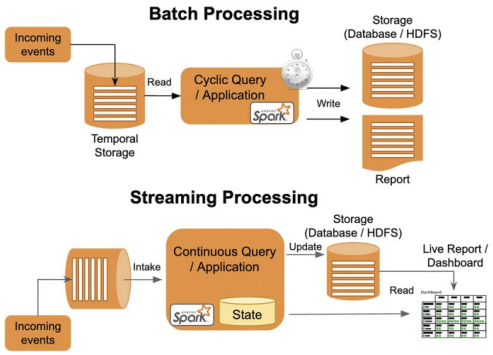


Figure 13: Micro-batch model: the stream is cut into small Spark batch jobs.

Two layers of work share this cluster. The *streaming layer* is the long-running ingestion job: it consumes all three Kafka topics, fixes latin1-mojibake on text fields, derives time columns, and appends each micro-batch to bronze Hudi tables (`staging.<topic>`). The *batch layer* is everything downstream of bronze – dbt and Superset: dbt runs on the same Spark cluster as the streaming job, reading the bronze Hudi tables and writing the silver and gold Hudi tables, and Superset connects to the Spark Thrift Server to query the gold tables for visualization.

Structured Streaming does not process events one at a time. Instead, as Figure 13 shows, the driver repeatedly polls the three Kafka topics for new offsets, groups the newly available records into a micro-batch, and schedules that micro-batch as a short-lived Spark job whose tasks run on the worker executors, in the same way a batch job would.

Structured Streaming resumes from a persisted checkpoint volume (`spark-checkpoints`) on restart; each micro-batch also commits its Kafka offsets to a `streamify-<topic>` consumer

group purely so lag is visible in Grafana (Spark itself never depends on that commit to resume).

4.5 Transformation Layer



Figure 14: Star schema: `fact_streams` and five dimensions.

The transformation layer turns each bronze Hudi table into an analytical data model over two further medallion layers, silver and gold (Section 2.4), so downstream queries never touch bronze directly:

- **Silver – dimension tables.** Five dimension tables built from bronze: `dim_users`, `dim_songs`, `dim_artists`, `dim_locations`, and `dim_datetime`. Each cleans and conforms one entity out of the raw bronze events, independent of any one fact table.
- **Gold – star schema.** The fact table `fact_streams`, built from bronze joined against the five silver dimensions. This is the normalized, storage-efficient model (Section 2.4).
- **Gold – wide table (`wide_streams`).** A denormalized table built by pre-joining `fact_streams` with all five silver dimension tables, so that Superset can query a single flat table instead of performing joins at dashboard-query time.



Figure 15: dbt DAG: bronze sources, star schema, `wide_streams`.

Instead of hand-written Spark jobs, dbt [5] generates the Spark SQL and submits it through the Spark Thrift Server (`method: thrift`), materializing every silver and gold model as a COPY_ON_WRITE Hudi table. The fact table, the wide table and the user and location dimensions are incremental models that `merge` new rows by a unique key; the other dimensions are rewritten per partition with `insert_overwrite`. dbt builds a DAG of models – declared in

`dbt_project.yml`, with each model a SQL query under `models/` – where every model transforms one or more upstream models into a new table. Figure 15 shows the resulting DAG, with bronze sources at the bottom, the silver dimension tables next, the gold `fact_streams` after that, and `wide_streams` at the top.

Why dbt, and why both a star schema and a wide table. dbt declares model dependencies explicitly (via `ref()`/`source()`), runs built-in tests (uniqueness, not-null, referential integrity) and generates documentation from the same files, which matters for a fact table that depends on five dimensions. The star schema keeps normalization and storage efficiency, while `wide_streams` keeps BI queries simple.

4.6 Serving Layer

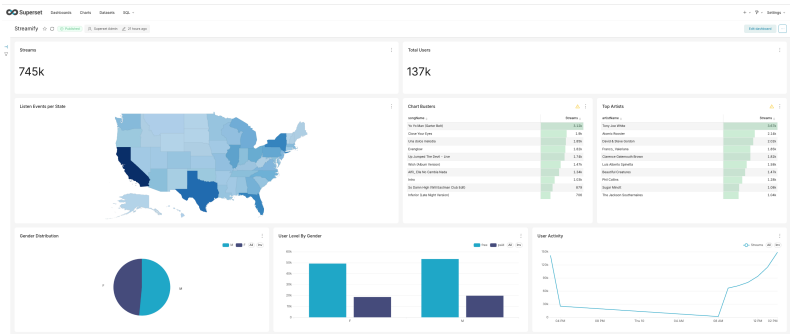


Figure 16: Superset dashboard on `wide_streams`, queried via Spark Thrift Server.

Apache Superset [3] connects to the same Spark Thrift Server that dbt submits SQL through, querying the `wide_streams` model to visualize insights through interactive dashboards (`superset/provisi` provisions them via the REST API). Figure 16 shows the resulting dashboard layout.

4.7 Orchestration Layer

Streaming runs all the time, but the rest of the pipeline has to be started by something: dbt must rebuild the models, the dashboard must pick up the new columns, and the Hudi tables need maintenance from time to time. At first these steps were run by hand, which is easy to forget. Apache Airflow 3 now does this. It runs as one container in standalone mode, which is enough for a single-machine demo. The DAGs are unpaused when created, so the schedules start as soon as the stack is up.

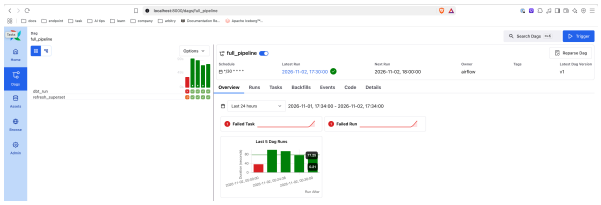


Figure 17: `full_pipeline` DAG, every 30 min. The red run failed before the seed fix.

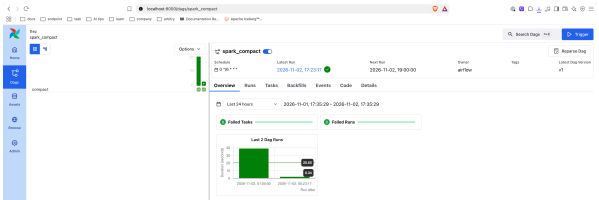


Figure 18: `spark_compact` DAG, every 6 h, one `compact` task.

There are three DAGs (Figures 17 and 18). `full_pipeline` runs every 30 minutes and has two tasks. The first, `dbt_run`, loads the `state_codes` seed and then runs all dbt models. The seed step is needed because `dbt run` does not create seed tables, and `dim_locations` reads from one. The second task, `refresh_superset`, first runs the dashboard provisioning script and then refreshes the dataset columns. The provisioning script is safe to repeat: it creates the dataset and dashboard only when they are missing. We added it after a fresh stack failed with "dataset not found", because the refresh ran before anything had created the dataset.

`spark_compact` runs every 6 hours and calls Hudi's `run_compaction` for each of the three bronze MOR tables. Hudi's default lock provider needs an atomic file create, which S3A does not offer, so the first runs failed with an "unsupported scheme" error. The job now passes an in-process lock provider in the call options. This lock only protects against other writers inside the Thrift Server, not against the streaming job, which is a separate process; a production system would need a Hive-metastore or ZooKeeper lock. Today every call returns an empty list, because the streaming writer only adds new Parquet files and never creates the log files that compaction merges (Section 4.2). The job is ready, but has real work only once updates or deletes reach a bronze table. The third DAG, `dbt_adhoc`, is started by hand and shows a small form where the user picks the dbt command, the model selection and whether to do a full refresh, so one-off runs need no code change.

All three DAGs send SQL to the same Spark Thrift Server instead of starting their own Spark applications, since the cluster has no free core for another one (Section 4.4). Two of them at once would compete for the Thrift Server's 2 cores, so they share an Airflow pool called `thrift_pool` with a single slot and take turns. dbt itself is installed in its own virtual environment inside the Airflow image, with its packages fetched at image build time.

4.8 Monitoring Layer



Figure 19: Grafana Kafka dashboard: consumer lag and rate per topic.



Figure 20: Grafana Spark dashboard: job metrics, JVM heap, container CPU/memory.

Three exporters feed one Prometheus instance: kafka-exporter (consumer-group lag per partition), Spark's metrics endpoint (streaming/batch counters, JVM heap) and cAdvisor (per-container CPU and memory). Grafana renders them in Figures 19 and 20.

Chapter 5

Conclusion

5.1 Results Achieved

This project achieved its core goal: understanding, first-hand, how a modern streaming platform works end to end. Building the pipeline meant learning why a Kafka partition needs a follower replica, why Structured Streaming's checkpoint must be separate from the consumer-group offset it also commits, when MERGE_ON_READ is worth its read-time merge cost over COPY_ON_WRITE, and how a fixed, shared compute budget shapes every downstream decision.

It also applied the course's distributed-systems concepts – partitioning and replication, checkpointed exactly-once processing, and consumer-lag observability – to a real system. In practice, the streaming lakehouse closes the gaps from Section 3.1: no duplicated lambda speed layer, a transactionally updatable lake, and independent storage and compute. The whole stack comes up reproducibly from one Docker Compose command.

5.2 Limitations

- **Small, fixed-size cluster.** The Spark cluster has only 4 cores, shared by the streaming job and the Thrift Server used by dbt, so there is no headroom for ad-hoc Spark SQL.
- **No autoscaling.** Kafka brokers and Spark workers are fixed at deployment time.
- **Synthetic dataset.** Eventsim data has not been validated against real traffic patterns or data-quality issues.

5.3 Future Directions

- Validate the pipeline with a real or real-scale streaming dataset.
- Add autoscaling for Kafka and Spark (e.g. via a container orchestrator).
- Add automated alerts (consumer lag, job failures) and a silver layer between bronze and gold.

References

[1] Apache Software Foundation. *Apache Hudi Documentation*. 2024. URL: https://hudi.apache.org/docs/overview (visited on 09/12/2026).

[2] Apache Software Foundation. *Apache Kafka Documentation*. 2024. URL: https://kafka.apache.org/documentation/ (visited on 09/12/2026).

[3] Apache Software Foundation. *Apache Superset Documentation*. 2024. URL: https://superset.apache.org/docs/intro (visited on 09/12/2026).

[4] Apache Software Foundation. *Structured Streaming Programming Guide*. 2024. URL: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html (visited on 09/12/2026).

[5] dbt Labs. *dbt Documentation*. 2024. URL: https://docs.getdbt.com/ (visited on 09/12/2026).

[6] MinIO, Inc. *MinIO Documentation*. 2024. URL: https://min.io/docs/minio/linux/index.html (visited on 09/12/2026).